JAVA SDE-2 INTERVIEW PREPARATION SERIES

STUDY STEP 18E OF 18 - PART E OF J

Advanced Java E: Performance, Diagnostics, and GC Incidents

Measurement, JMH, jcmd, JFR, Thread Dumps, Heap Evidence, and Tuning Playbooks

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Move from symptoms to evidence using measurement design, JMH, runtime tools, recordings, dumps, leak analysis, GC reasoning, and safe mitigation.

JMH | JCMD | JSTACK | JFR | HEAP | GC | INCIDENTS

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `Study Step 18D – Advanced Java D – Concurrency`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Part E teaches an evidence-first diagnostic method so performance claims and tuning actions remain falsifiable and operationally safe.

Readiness map

Decision	Evidence
START HERE	Latency, throughput, CPU, allocation, or pause behavior changed; A benchmark lacks warmup or consumption; A dump or recording must distinguish competing hypotheses; A proposed JVM flag change lacks evidence.
PREPARE FIRST	Study Step 18A runtime model; Study Step 18D concurrency recommended.
MOVE ON WHEN	Design meaningful performance measurements; Use core JDK diagnostic tools; Analyze leaks and GC incidents; Propose reversible mitigations and prevention.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Study Path Position

18D - Advanced Java D - Concurrency	**YOU ARE HERE** **18E - Advanced Java E - Diagnostics**	18F - Advanced Java F - Backend

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Start Here - Your Route Through This Volume 2

Part I - Core Learning 4

Chapter 1 - Performance Methodology and JMH Benchmarking 4

Chapter 2 - JVM Diagnostics with jcmd, jstack, jmap, JFR, and Mission Control 14

Chapter 3 - Memory Leaks, GC Incidents, and Tuning Playbooks 24

Chapter 4 - JVM Tools, Flags, and Incident Commands 34

Part II - Practice and Handoff 44

Practice Ladder and Next Steps 44

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Performance Methodology and JMH Benchmarking

Learning objectives

By the end of this chapter, you should be able to:

- turn a vague performance concern into a falsifiable question and a measurable objective;
- distinguish latency distributions, throughput, utilization, allocation, and asymptotic cost;
- explain warmup, tiered compilation, dead-code elimination, constant folding, and benchmark contamination;
- build a disciplined JMH benchmark with state, forks, warmup, measurement, and parameters;
- interpret measurements with uncertainty instead of selecting one attractive number; and
- connect microbenchmark evidence to profiling, load tests, production traces, and business constraints.

WHY IT WORKS | Why this matters at SDE-2

Performance work at SDE-2 is decision work. The engineer must identify which user-visible objective is failing, collect evidence at the correct layer, and make the smallest change that improves the limiting resource without violating correctness. "This collection is faster" or "the JVM will optimize it" is not an engineering argument.

Java makes naive measurement particularly misleading. Code begins interpreted or lightly compiled, hot methods may be recompiled, unused work can disappear, object allocations can be scalar-replaced, garbage collection can interrupt samples, and the operating system can move the process between CPUs. JMH handles many mechanics, but it cannot repair a meaningless workload or prove that a micro-level improvement changes end-to-end behavior.

WHY IT WORKS | First-principles model

Performance is work completed per constrained resource and time. Begin with an observable target:

```text
Question: Why did checkout p99 latency rise from 180 ms to 420 ms?
Scope: production requests in region A after release R
Constraints: error rate <= 0.1%, same durability and validation
Evidence: latency histogram, traces, CPU, allocation, GC, downstream timings
Hypothesis: repeated JSON materialization increased allocation and GC pauses
Experiment: remove one copy, compare controlled canary and allocation profile
```

The feedback loop is:

```text
define -> measure baseline -> form hypothesis -> design experiment
       -> change one variable -> validate -> deploy safely -> observe
```

Latency is a distribution, not one scalar. Throughput and latency interact through queueing: as utilization approaches capacity, waiting time can rise sharply. A faster isolated operation may not improve the system if a database, lock, network call, or admission limit remains the bottleneck.

> **Specification boundary:** Java SE does not specify JIT compilation thresholds, escape analysis outcomes, code layout, garbage collector ergonomics, or JMH behavior. These are implementation and tool concerns. Preserve Java semantics, but measure optimization behavior on the actual JDK, collector, hardware, and configuration.

Core terminology

- **Throughput:** Completed operations per unit time.
- **Latency:** Time for one operation; report percentiles and the measurement window.
- **Tail latency:** High-percentile latency such as p95, p99, or p99.9.
- **Utilization:** Fraction of a resource's available capacity in use.
- **Service time:** Active processing time excluding some or all queue waiting, depending on definition.
- **Warmup:** Execution before measurement so runtime state approaches the intended regime.
- **Fork:** Fresh JVM process used as an independent benchmark run.
- **Steady state:** Period whose relevant runtime behavior is sufficiently stable for the question.
- **Dead-code elimination:** Removal of computations whose results cannot affect observable behavior.
- **Constant folding:** Compile-time or JIT-time evaluation of expressions known to be constant.
- **Escape analysis:** Reasoning that can enable allocation elimination or synchronization optimization.
- **Benchmark state:** Data whose lifecycle and sharing are controlled by the harness.
- **Confidence interval/error estimate:** Range describing measurement uncertainty under a statistical model.
- **Coordinated omission:** Missing slow observations because the load generator waits before issuing the next request.

Detailed mechanics

Define the metric before the experiment

State the operation, population, units, percentile, traffic shape, input distribution, and correctness constraints. "Average endpoint time" can hide a damaging tail. CPU time excludes blocking. Wall time includes scheduling and waiting. Allocation rate does not directly equal retained heap. Pick metrics tied to the hypothesis.

Record a baseline and a control. Change one primary variable. Preserve source data, JVM arguments, dependency versions, CPU limits, and test duration when possible. Run enough independent trials to see variance. If a result changes sign across forks, the honest conclusion is uncertainty, not the best-looking fork.

Why stopwatch loops fail

`System.nanoTime` is suitable for elapsed intervals but does not create a correct harness. A hand-written loop often measures compilation transitions, loop machinery, a constant expression, timer calls, or shared setup. It can also omit the computed result, allowing the optimizer to remove the work.

The JIT optimizes the program it sees. If benchmark inputs are constants, it may precompute results. If an allocated object does not escape, it may never become a heap object. These are valid production

optimizations, but a benchmark intended to measure allocation or general inputs must prevent accidental specialization without preventing realistic optimization.

JMH execution model

JMH, the Java Microbenchmark Harness, is a separate tool rather than a Java SE API. Its generated harness controls invocation, warmup, measurement, result consumption, and process forks.

Important annotations include:

- `@Benchmark` marks measured methods.
- `@BenchmarkMode` selects throughput, average time, sample time, or single-shot behavior.
- `@OutputTimeUnit` chooses presentation units.
- `@Warmup` and `@Measurement` configure iteration phases.
- `@Fork` requests fresh JVM processes.
- `@State` defines state sharing: per thread, benchmark instance, or group.
- `@Param` expands controlled input cases.
- `@Setup` and `@TearDown` manage state outside timed invocations at trial, iteration, or invocation level.

Use multiple forks for JVM-level independence. Warmup is not "always five iterations"; it must be long enough for the workload and question. Inspect per-iteration results for drift. Single-shot mode is appropriate for cold or one-off operations only when that is explicitly the target.

Preventing invalid optimization without disabling the JVM

Return the result from the benchmark or pass it to JMH's `Blackhole`. Do not add arbitrary logging, volatile writes, or synchronization merely to keep work alive; they can dominate the measurement. Supply varied state through parameters or setup when constants are unrealistic.

Move construction out of timed code unless construction is the operation under test. If each invocation mutates state, reset at a deliberate lifecycle level and include or exclude reset cost according to the real question. Invocation-level setup can be expensive enough to distort very small operations.

Benchmark modes and profilers

Throughput mode answers operations per time. Average time reports mean operation time under the harness model. Sample time records a sample distribution; it is not a replacement for a production latency histogram. Single-shot measures individual invocations and is sensitive to noise.

JMH profilers can add allocation, GC, stack, compiler, or platform-counter evidence, but profilers perturb execution. Compare profiled runs with unprofiled runs. Hardware counters and assembly views depend on OS permissions, architecture, installed tools, and JVM support.

> **HotSpot note:** Tiered compilation, inlining budgets, on-stack replacement, escape analysis, and code-cache behavior are HotSpot implementation details and version-sensitive. A benchmark result from Java 17 is evidence for that setup, not a promise for Java 21 or another JVM.

From micro to macro

A microbenchmark isolates mechanism. A component benchmark includes serialization, pools, and local dependencies. A load test checks queueing and capacity. Production observation validates actual inputs, contention, and downstream behavior. Use the lowest layer that can answer the question, then validate upward.

Do not extrapolate nanoseconds saved per isolated call by multiplying by request count unless the operation is actually on the critical path at that frequency. Profiling should confirm contribution to CPU or latency. Optimization can shift cost to memory, startup, code size, or maintainability.

TRACE IT | Worked Java example

This JMH benchmark compares miss lookup in a list and a set. It isolates lookup by constructing state once per trial:

JAVA (continued 1/2)

```java
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import java.util.concurrent.TimeUnit;
import java.util.stream.IntStream;
import org.openjdk.jmh.annotations.Benchmark;
import org.openjdk.jmh.annotations.BenchmarkMode;
import org.openjdk.jmh.annotations.Fork;
import org.openjdk.jmh.annotations.Level;
import org.openjdk.jmh.annotations.Measurement;
import org.openjdk.jmh.annotations.Mode;
import org.openjdk.jmh.annotations.OutputTimeUnit;
import org.openjdk.jmh.annotations.Param;
import org.openjdk.jmh.annotations.Scope;
import org.openjdk.jmh.annotations.Setup;
import org.openjdk.jmh.annotations.State;
import org.openjdk.jmh.annotations.Warmup;

@BenchmarkMode(Mode.AverageTime)
@OutputTimeUnit(TimeUnit.NANOSECONDS)
@Warmup(iterations = 5, time = 1)
@Measurement(iterations = 7, time = 1)
@Fork(3)
public class MembershipBenchmark {
    @State(Scope.Thread)
    public static class Data {
        @Param({"16", "1024", "65536"})
```

JAVA (continued 2/2)

```
        int size;

        List<Integer> list;
        Set<Integer> set;
        Integer missing;

        @Setup(Level.Trial)
        public void setup() {
            ArrayList<Integer> values = IntStream.range(0, size)
                    .boxed()
                    .collect(java.util.stream.Collectors.toCollection(ArrayList::new));
            list = values;
            set = new HashSet<>(values);
            missing = -1;
        }
    }

    @Benchmark
    public boolean listMiss(Data data) {
        return data.list.contains(data.missing);
    }

    @Benchmark
    public boolean setMiss(Data data) {
        return data.set.contains(data.missing);
    }
}
```

The JMH annotation processor and runtime must be configured in the build. Pin their version rather than copying an unversioned command from another project. A typical generated benchmark artifact can be filtered by benchmark name and optionally run with a GC profiler, but exact command-line options belong to the selected JMH version.

TRACE IT | Execution or memory walkthrough

For each fork, JMH starts a new JVM. For each parameter value it creates thread-scoped `Data`, runs trial setup, then executes warmup iterations whose results are not reported as measurements. Compilation and profile information evolve during warmup. Measurement iterations follow and JMH aggregates fork-level observations.

For `size = 16`, `list.contains(-1)` checks 16 boxed integer references/equality operations. The hash set computes a hash, selects a bucket, and usually confirms absence after a small amount of work, but it has a larger structure and less locality. At tiny sizes, constants may make the list competitive. At 65,536 elements, the list miss is linear while expected set lookup remains constant under good hashing.

The benchmark returns a Boolean result to the harness, preventing trivial removal. It does not time collection construction, iteration, memory footprint, cache warmness across production requests, successful lookups at varied positions, or concurrent access. Those are separate experiments.

Complexity and performance

The underlying algorithm predicts trends:

Operation	List	Hash set, typical expectation
miss lookup	`O(n)`	expected `O(1)`
build from `n` values	`O(n)`	expected `O(n)`
storage	`O(n)` references	`O(n + capacity)` plus entry overhead
ordered iteration	insertion sequence for list	no general hash-set order

Big-O does not predict the crossover size. Hash computation, equality, allocation, cache locality, table load, JVM optimization, and hardware determine constants. The benchmark measures one point in this design space.

Measurement cost also matters. More forks and longer iterations improve confidence but consume time. Prefer enough independent evidence to make the decision, not maximal duration by ritual. Report configuration, sample counts, central estimates, uncertainty, outliers, and all tested parameter values. Never report more precision than noise supports.

WATCH OUT | Edge cases and common mistakes

- Optimizing without a baseline, SLO, profile, or explicit hypothesis.
- Reporting only averages and hiding tail behavior or errors.
- Timing startup while claiming steady-state throughput, or warming up while claiming cold startup.
- Leaving benchmark results unused and measuring code eliminated by the optimizer.
- Putting data generation, logging, assertions, or random seeding in the timed operation unintentionally.
- Sharing mutable benchmark state across threads without matching production semantics.
- Running one fork and treating one score as universal.
- Comparing runs with different CPU quotas, power modes, thermal states, JVM flags, or background load.
- Using unrealistic constant inputs that enable specialization.
- Treating a profiler run as unperturbed timing evidence.
- Assuming a statistically significant micro improvement is operationally meaningful.
- Running load generators that suffer coordinated omission and under-report queue delay.
- Claiming causality from correlation in one production graph.

Production engineering notes

Start with production-safe telemetry: request histograms, traces, error rate, CPU, allocation, GC, pool saturation, database timing, and queue depth. Align timestamps and release markers. Sampling profilers are usually safer than ad hoc high-frequency instrumentation, but every diagnostic has cost.

Benchmark on the same major JDK, collector, architecture, and container limits as production. Pin benchmark code and dependencies in source control. Capture JVM arguments and environment metadata with results. Run noisy-neighbor and concurrency tests when shared infrastructure is part of the question.

Optimize one confirmed hot path, add correctness and performance regression tests at the appropriate layer, and canary the change. A microbenchmark threshold in CI can be flaky; compare distributions on controlled runners and alert on substantial, repeated regressions rather than nanosecond drift.

Stop when the SLO and capacity target are met with margin. Further optimization spends complexity budget and can reduce reliability. Preserve a written experiment log so the next incident does not repeat disproven hypotheses.

TRY IT | Interview questions and model answers

Why is a loop around `System.nanoTime` not enough?

It does not control warmup, forks, dead-code elimination, setup, or statistical independence. Timer and loop overhead can dominate. JMH supplies a generated harness, but the workload still needs a valid question and realistic inputs.

Why does JMH use warmup and forks?

Warmup lets runtime compilation and profiles approach the target regime. Forks create fresh JVM processes, reducing dependence on one process's compilation, heap, and code-cache history.

How do you prevent dead-code elimination?

Return the computed result or consume it through JMH's `Blackhole`. Also avoid fully constant inputs if the production operation receives varied data.

When is a microbenchmark the wrong tool?

When the question depends on I/O, queueing, thread-pool saturation, distributed calls, or user-visible tail latency. Use component, load, or production evidence for those effects.

Can parallel code have higher throughput but worse latency?

Yes. It can increase contention, coordination, queueing, allocation, and tail variation while completing more aggregate work. Measure both under the intended concurrency.

What would you report with a benchmark result?

Question, code revision, JDK and arguments, hardware and limits, JMH configuration, inputs, forks, score and uncertainty, profiler observations, correctness checks, and the scope of conclusions.

TRY IT | Exercises

1. Design a JMH benchmark comparing string concatenation strategies for 10, 100, and 10,000 fragments. Decide whether construction belongs inside the timed method.
2. Find three ways a benchmark of `Math.log(42.0)` could measure constant folding rather than general logarithm cost.
3. Write separate experiments for cold startup and warmed request throughput. Explain why one configuration cannot answer both.
4. Given p50 20 ms, p99 900 ms, low CPU, and a saturated connection pool, form three hypotheses and choose evidence for each.
5. Add successful first, middle, and last list lookups to the worked example without introducing random-number generation into timed code.
6. Review a claimed 3 percent speedup whose fork error intervals overlap substantially. Write the appropriate conclusion and next experiment.

RECAP | Chapter summary

Performance engineering begins with a user-visible objective, a baseline, and a falsifiable hypothesis. Java runtime optimization makes naive timing unreliable. JMH controls microbenchmark lifecycle, result consumption, warmup, measurement, and forks, but it cannot supply realistic semantics. Use complexity to predict trends, profilers to locate mechanisms, higher-level tests to expose queueing and integration effects, and production canaries to validate impact. Report uncertainty and stop when measured objectives are met.

TRY IT | Revision checklist

- [ ] I define the operation, distribution, metric, load, and correctness constraints before measuring.
- [ ] I distinguish throughput, latency percentiles, service time, utilization, allocation, and retention.
- [ ] I can explain warmup, tiered compilation, dead-code elimination, constant folding, and escape analysis.
- [ ] I use JMH state, setup, parameters, measurement iterations, and forks intentionally.
- [ ] I consume benchmark results and isolate setup according to the question.
- [ ] I report environment, uncertainty, outliers, and scope of conclusions.
- [ ] I validate microbenchmark findings with profiles and the appropriate higher-level experiment.
- [ ] I label HotSpot and tool behavior as version-sensitive.

SDE-2 CORE CHAPTER

Chapter 2 - JVM Diagnostics with jcmd, jstack, jmap, JFR, and Mission Control

Learning objectives

By the end of this chapter, you should be able to:

- build a timestamped incident timeline before changing JVM state;
- select low-risk diagnostics for CPU, latency, deadlock, allocation, heap, and native-memory symptoms;
- use `jcmd`, thread dumps, Java Flight Recorder, and heap tools with explicit cost and data-handling controls;
- interpret thread states, repeated stacks, JFR events, histograms, and heap dumps without overclaiming causality;
- explain when `jstack`, `jmap`, and Mission Control are appropriate; and
- write a safe evidence-capture runbook for Java 17 and Java 21 deployments.

WHY IT WORKS | Why this matters at SDE-2

During an incident, diagnostic commands are production changes. Some briefly stop application threads, some walk the heap, some write files as large as the heap, and many capture secrets. An SDE-2 engineer must gather enough evidence to distinguish CPU saturation, lock contention, garbage collection, downstream waiting, native-memory growth, and deadlock while avoiding a second outage caused by diagnosis.

The strongest incident report correlates sources. A thread dump is a moment, a JFR recording is a timeline, service metrics show impact, and a heap dump shows one graph. None alone proves root cause. Preserve timestamps, process identity, container identity, release version, JVM configuration, host pressure, and command results so observations can be aligned.

WHY IT WORKS | First-principles model

Diagnostics answer three questions:

```text
What is the symptom?
Which resource or dependency is limiting progress?
What evidence would disprove each plausible cause?
```

Use an escalation ladder:

```text
existing metrics/logs/traces
    -> process identity and JVM metadata
    -> short JFR or repeated thread dumps
    -> targeted histograms/native-memory summaries
    -> heap dump or invasive tooling only with capacity and approval
```

Start with evidence already being collected because it adds no incident-time perturbation. Prefer time-bounded, reversible capture. Do not tune, restart, force GC, or clear caches before obtaining a baseline unless immediate recovery takes priority over diagnosis. If recovery is necessary, say which evidence was lost.

> **Specification boundary:** These tools are JDK and vendor facilities, not Java Language Specification guarantees. Command names, accepted arguments, attach behavior, event fields, virtual-thread representation, and overhead can change between JDK builds. Inspect `jcmd <pid> help` on the target runtime and test runbooks on the exact distribution.

Core terminology

- **Attach:** Mechanism by which a diagnostic process connects to a running JVM.
- **Safepoint:** Runtime state where selected VM operations can safely inspect or modify global state.
- **Thread dump:** Snapshot of Java threads, states, stacks, and optionally owned synchronizers.
- **JFR:** Java Flight Recorder, an event recording facility integrated with supported JDKs.
- **JMC:** Java Mission Control, a graphical analysis application commonly used with JFR recordings.
- **Class histogram:** Counts and shallow bytes grouped by class.
- **Heap dump:** Snapshot of heap objects and references, commonly in HPROF form.
- **Native memory:** Process memory outside ordinary Java heap, including metaspace, code, threads, direct buffers, and native libraries.
- **NMT:** HotSpot Native Memory Tracking.
- **Deadlock:** Cycle in which participants wait indefinitely for resources held by one another.
- **Perturbation:** Change in application behavior caused by measurement.
- **Evidence custody:** Secure storage, access control, retention, and deletion of diagnostic artifacts.

Detailed mechanics

Establish identity and context

Confirm the operating-system PID inside the relevant PID namespace. In containers, a host PID and container PID can differ. Record current time with timezone, pod or host, deployment revision, traffic, and whether the symptom is active. Avoid relying solely on `jps`; not every Java process is discoverable in every namespace or permission setup.

`jcmd <pid> VM.version`, `VM.command_line`, and `VM.flags` can capture runtime identity and configuration. System properties and command lines may contain credentials, tokens, paths, or customer identifiers, so collect them only into restricted storage. `jcmd <pid> help` lists commands actually available in that JVM.

Thread dumps as repeated snapshots

`jcmd <pid> Thread.print -l` is the preferred general HotSpot route in many modern deployments. `jstack -l <pid>` is a useful fallback when available and supported. Capture several dumps separated by a meaningful interval. A repeated identical stack on an on-CPU method is stronger evidence than one appearance.

Common thread states require context:

- `RUNNABLE` means eligible/running from the JVM perspective; it can include native I/O and does not prove CPU consumption.
- `BLOCKED` means waiting to enter an intrinsic monitor.
- `WAITING` can be normal parking in a pool, queue, or `join`.
- `TIMED_WAITING` includes timed sleep, park, and wait.
- terminated threads do not appear as live work.

Look for deadlock reports, many threads blocked behind the same owner, pool threads waiting on downstream calls, and stacks that remain active across captures. Correlate native thread IDs with OS CPU tools where supported. Thread names are operational API: name pools and critical workers meaningfully.

Virtual threads change scale. Dump formats and commands capable of representing large virtual-thread sets have evolved across JDK releases. Do not assume one platform-thread-style dump will list or interpret every virtual-thread task identically; validate Java 21 tooling and prefer JFR events or supported virtual-thread dumps for the target build.

Java Flight Recorder

JFR records timestamped JVM and application events with configurable settings. Useful event families include execution samples, allocation samples, garbage collection, monitor contention, thread parking, file/socket I/O, exceptions, class loading, and JVM configuration. Availability and fields are version-dependent.

A continuously running, size- and age-bounded recording is often more valuable than starting after an event. It preserves the lead-up. If continuous recording is not configured, `jcmd` can start a named,

duration-bounded recording and write it to a restricted path. The `default` configuration targets lower overhead; `profile` commonly collects more detail at higher cost. Validate both overhead and disk behavior under load.

Typical command shapes are:

BASH

```
jcmd 12345 JFR.check
jcmd 12345 JFR.start name=incident settings=default duration=120s filename=/restricted/incident.jfr
jcmd 12345 JFR.dump name=incident filename=/restricted/snapshot.jfr
jcmd 12345 JFR.stop name=incident
```

Do not paste these blindly. Confirm PID, available help, output directory, free space, permissions, recording name, and the JDK's syntax. JFR can contain class names, paths, stack traces, thread names, URLs, and application event fields.

Mission Control opens recordings for timeline analysis, event browsing, automated rules, flame-like views, lock analysis, and GC inspection. It is an analyzer, not an oracle. Automated warnings are leads to correlate with service impact. JMC versions are distributed separately from some JDKs and should be compatible with the recording format in use.

Class histograms and heap dumps

A class histogram answers "which classes account for many live or heap-visible objects and shallow bytes?" It does not show why they are retained. `jcmd <pid> GC.class_histogram` and `jmap -histo` variants can be costly because implementations may require a safepoint and heap traversal; live-only variants can trigger additional GC work. Syntax and impact vary.

A heap dump preserves objects and reference edges for dominator and path-to-root analysis. It can pause the process, perform substantial I/O, consume disk near heap size or more, expose secrets, and worsen a memory-starved host. Verify disk headroom, storage performance, encryption/access controls, and whether a replica can be removed from traffic. Prefer `jcmd` heap-dump facilities over older `jmap` paths when the target JDK recommends them.

Configure `-XX:+HeapDumpOnOutOfMemoryError` and a controlled dump path only after validating disk, permissions, retention, and restart behavior. The JVM may be too impaired to complete a dump. Never make heap-dump success the only OOM evidence plan.

Native memory and process evidence

Resident-set growth with stable heap may come from thread stacks, direct buffers, metaspace, code cache, JNI, memory mapping, allocators, or kernel accounting. Process metrics and OS maps provide evidence. On HotSpot, NMT can categorize tracked native allocations, but it must be enabled at process startup and adds overhead. Summary tracking is normally less costly than detail tracking.

With NMT enabled, `jcmd <pid> VM.native_memory baseline` followed later by `summary.diff` or the target version's equivalent can expose category growth. NMT does not track every native allocation and its totals need not equal RSS. Treat categories as a model to correlate, not exact process accounting.

> **HotSpot note:** Attach implementation, safepoint behavior, NMT categories, JFR event sets, compiler names, and diagnostic-command side effects are HotSpot and build specific. Container security policies can disable attach even when commands are present.

Tool failure is evidence, not permission to escalate blindly

Attach can fail because of permissions, namespaces, disabled mechanisms, a hung JVM, or incompatible tools. Use tools from the same JDK family as the target. Do not reach immediately for forced serviceability agents or debuggers on production; they can suspend or destabilize the process. Follow an approved runbook and decide whether replica failover, core dump, or restart is safer.

TRACE IT | Worked Java example

This bounded program creates observable intrinsic-lock contention without deadlocking forever:

JAVA

```java
import java.util.ArrayList;
import java.util.List;

public final class ContentionDemo {
    private static final Object LOCK = new Object();

    private static void work() {
        for (int i = 0; i < 100; i++) {
            synchronized (LOCK) {
                try {
                    Thread.sleep(20);
                } catch (InterruptedException interrupted) {
                    Thread.currentThread().interrupt();
                    return;
                }
            }
        }
    }

    public static void main(String[] args) throws InterruptedException {
        List<Thread> workers = new ArrayList<>();
        for (int i = 0; i < 4; i++) {
            Thread worker = new Thread(ContentionDemo::work, "worker-" + i);
            workers.add(worker);
            worker.start();
        }
        for (Thread worker : workers) {
            worker.join();
        }
    }
}
```

In a disposable local environment, run the process, find its PID safely, and capture two thread dumps while it is active. A short JFR recording can then show monitor-blocked events. Do not run an artificial contention program on a shared production host.

TRACE IT | Execution or memory walkthrough

At most one worker owns `LOCK`. That worker calls `sleep` while still inside the synchronized block, so it appears `TIMED_WAITING` while retaining the monitor. Other workers attempting entry appear `BLOCKED`, and the dump identifies the monitor owner when lock detail is available.

Twenty milliseconds later, the owner wakes and exits. Scheduling and monitor acquisition determine the next owner; intrinsic locks do not promise strict FIFO fairness. Across several dumps, different workers may own the monitor, but the structural pattern remains one sleeper and multiple blocked contenders.

A JFR timeline adds duration: it can reveal how long monitor entry was blocked and which stack requested it. OS CPU may remain low because most threads wait. This combination disproves the hypothesis that high latency necessarily means CPU saturation.

No deadlock exists because the owner eventually releases one lock and there is no dependency cycle. A single dump with blocked threads is therefore not a deadlock diagnosis.

Complexity and performance

Diagnostic cost scales with captured scope:

Evidence	Typical relative impact	Scaling concern
existing metrics/logs	already paid	cardinality and logging volume
thread dump	usually brief	number of threads and stack depth
low-overhead JFR	designed for continuous use	event rate, stack traces, disk limits
profile JFR	higher	sampling/event configuration and workload
class histogram	moderate to high	heap object count and safepoint work
heap dump	high	live/used heap, pause, disk and I/O bandwidth
detailed NMT	startup-enabled overhead	native allocation rate and tracking detail

These are not universal rankings. A JVM with millions of virtual threads, a nearly full disk, or a stalled safepoint can change risk. Estimate artifact size and latency, test on a realistic replica, and define an abort condition.

Analysis complexity also matters. A histogram is small enough for quick comparison but lacks edges. A heap dump provides retention paths but can take significant offline processing memory. JFR duration and event settings trade history against file size and detail.

WATCH OUT | Edge cases and common mistakes

- Attaching to the wrong PID or wrong container namespace.
- Collecting credentials from command lines, properties, heap, or JFR into world-readable files.
- Taking one thread dump and labeling every waiting thread a problem.
- Treating `RUNNABLE` as proof that a thread consumes CPU.
- Calling blocked threads a deadlock without a wait cycle.
- Forcing GC before preserving the heap behavior under investigation.
- Requesting a heap dump on a nearly full filesystem or latency-critical sole instance.
- Assuming a histogram's shallow bytes identify the retaining owner.
- Comparing NMT totals directly with RSS and declaring the difference a leak.
- Starting a high-detail recording without duration and size controls.
- Using diagnostic tools from an incompatible JDK distribution or version.
- Restarting before recording JVM arguments, timeline, and at least low-cost evidence.
- Uploading diagnostic artifacts to unapproved analysis services.
- Treating a Mission Control rule as proven causality.

Production engineering notes

Prepare diagnostics before incidents. Bake compatible tools into an approved debug image or host path, configure secure artifact storage, define who may attach, and test commands against Java 17 and Java 21 services. Keep commands parameterized by verified PID and never use broad kill or file targets.

Enable bounded continuous JFR where overhead tests permit. Give recordings unique incident IDs and UTC timestamps. Apply least privilege, encryption, short retention, and an audit trail. Heap dumps usually require the strongest classification because they can contain entire request bodies and credentials.

Capture three synchronized views: application impact, JVM behavior, and host/container limits. CPU throttling, memory limits, file descriptors, DNS, storage, and downstream pools can imitate JVM problems. Preserve deployment events and feature-flag changes.

Separate recovery from root-cause analysis. Traffic shedding, failover, or restart may be the correct first action. Record that decision, collect evidence from another replica or continuous recording, and reproduce later. Never delay recovery solely to obtain a perfect dump.

TRY IT | Interview questions and model answers

What would you collect for a Java latency spike?

First correlate request histograms, errors, traces, CPU/throttling, GC, allocation, pools, and downstream latency. Then verify process metadata and capture a short JFR or repeated thread dumps. Escalate to heap evidence only if the hypothesis requires it and impact is acceptable.

Why take multiple thread dumps?

One dump is a snapshot. Repeated dumps show whether the same stacks make no progress, which owners change, and whether contention or downstream waiting persists.

What is the difference between a histogram and a heap dump?

A histogram aggregates class counts and shallow bytes. A heap dump preserves individual objects and reference edges, enabling dominator and root-path analysis at much greater cost and sensitivity.

How do JFR and JMC relate?

JFR records timestamped events in the JVM. Mission Control analyzes recordings and presents timelines and rules. It helps form hypotheses but does not automatically prove root cause.

Can NMT prove a native-memory leak?

It can show growth in tracked HotSpot categories and guide a hypothesis. It omits some allocations and differs from RSS, so correlate it with process maps, direct-buffer metrics, thread counts, and repeated baselines.

Why can a diagnostic command be dangerous?

It can safepoint or pause the JVM, traverse a huge heap, consume disk and I/O, expose sensitive data, or fail under attach restrictions. Validate impact and storage before running it.

TRY IT | Exercises

1. Given high CPU and stable GC, design a capture sequence using OS per-thread CPU, repeated dumps, and JFR samples.
2. Given rising RSS but flat committed heap, list evidence for thread stacks, direct buffers, metaspace, and native libraries.
3. Write a runbook precheck for heap dumping a 24 GB service. Include replica status, disk, access, pause, and deletion.
4. Run the contention example locally and explain `BLOCKED`, `TIMED_WAITING`, monitor ownership, and why it is not deadlock.
5. Design a continuous JFR retention policy for a service with strict customer-data controls.
6. Review a thread dump with 200 idle pool workers in `WAITING`. Explain what additional evidence is required before tuning the pool.

RECAP | Chapter summary

JVM diagnosis is controlled evidence collection. Begin with timelines and existing telemetry, verify process identity, and escalate from repeated thread snapshots or bounded JFR to histograms and heap dumps only when the hypothesis justifies their cost. Thread state needs temporal and OS context. JFR provides event history; Mission Control supports analysis; heap and native-memory tools answer narrower retention questions. Every artifact is sensitive, every command is version-dependent, and recovery may take priority over perfect evidence.

TRY IT | Revision checklist

- [ ] I verify PID, namespace, JDK, timestamp, release, and symptom before attachment.
- [ ] I can interpret thread states without equating `RUNNABLE` with CPU or waiting with failure.
- [ ] I capture repeated dumps and correlate them with OS and request evidence.
- [ ] I can start, dump, and stop a bounded JFR recording after checking target help.
- [ ] I distinguish JFR recording from Mission Control analysis.
- [ ] I know the cost and information difference among histogram, heap dump, and NMT.
- [ ] I secure artifacts and check disk, replica capacity, and abort conditions.
- [ ] I label HotSpot, vendor, container, and JDK-version behavior explicitly.

SDE-2 CORE CHAPTER

Chapter 3 - Memory Leaks, GC Incidents, and Tuning Playbooks

Learning objectives

By the end of this chapter, you should be able to:

- distinguish high allocation, intended live data, heap retention leaks, native growth, and resource leaks;
- read heap occupancy, allocation, pause, CPU, promotion, and process-memory signals together;
- investigate a retention path from GC root to dominated objects;
- execute a staged memory or GC incident playbook without destroying evidence;
- tune only after defining latency, throughput, footprint, and headroom goals; and
- prevent common leaks through bounded ownership, lifecycle APIs, and observability.

WHY IT WORKS | Why this matters at SDE-2

An out-of-memory failure is an endpoint, not a root cause. A growing cache, an overloaded queue, a class-loader leak, direct-buffer growth, too many threads, or an undersized container can all end in memory failure and require different fixes. Increasing `-Xmx` can buy recovery time, hide an unbounded invariant, or make pauses and heap dumps larger.

SDE-2 engineers are expected to stabilize service, classify the memory domain, preserve evidence, and separate allocation pressure from unwanted retention. They must also resist tuning folklore. A collector flag is not a substitute for controlling cardinality, closing resources, removing listeners, bounding concurrency, and respecting the process memory limit.

WHY IT WORKS | First-principles model

Garbage collection reclaims objects that are not reachable from GC roots. A heap leak in a managed runtime is therefore unwanted reachability, not forgotten manual deallocation:

```text
GC root -> long-lived owner -> collection -> entry -> large object graph
```

The key quantities are:

```text
allocation rate = bytes allocated per unit time
live set        = bytes still reachable after an effective collection
headroom        = usable memory above live set for allocation and GC work
```

High allocation with a stable post-collection baseline is pressure, not necessarily a leak. A rising post-collection baseline under comparable load suggests increasing live data. Stable Java heap with rising process resident memory suggests a non-heap or native domain.

The investigation loop is:

```text
stabilize -> classify memory domain -> preserve timeline
          -> compare repeated evidence -> find owner/path
          -> fix lifecycle or capacity -> load-test -> canary -> monitor
```

> **Specification boundary:** Java specifies reachability concepts and reference types, not a collector algorithm, generation layout, region size, pause target, object age threshold, or out-of-memory message. Collector behavior and diagnostic counters are JVM/vendor/version specific.

Core terminology

- **GC root:** Starting point for reachability, such as selected thread, static, class-loader, JNI, or VM references.
- **Live set:** Objects that remain reachable after collection under the chosen observation.
- **Shallow size:** Memory occupied directly by one object.
- **Retained size:** Memory that could become reclaimable if an object were removed, under dominator analysis.
- **Dominator:** Object through which every root path to another object passes.
- **Allocation pressure:** High creation rate that forces frequent collection even if objects die quickly.
- **Promotion:** Movement or classification of surviving objects into longer-lived storage in a generational collector.
- **Fragmentation:** Free memory exists but not in a form suitable for requested allocation or collector progress.
- **Humongous/large object:** Collector-specific category for unusually large allocation.
- **Metaspace:** HotSpot native memory commonly used for class metadata.
- **Direct memory:** Native storage used by direct byte buffers and related I/O.
- **Leak slope:** Rate at which a stable-comparison memory baseline grows.
- **Backlog:** Work retained because production exceeds consumption.

Detailed mechanics

Classify before collecting expensive evidence

Start with container or host memory, Java heap used/committed/max, post-GC occupancy, allocation rate, GC pause and CPU, thread count, loaded classes, direct-buffer pool metrics, file descriptors, and queue/cache cardinality. Align them with traffic and deployment changes.

Useful patterns include:

- high allocation, frequent collections, stable baseline: allocation pressure;
- rising post-collection old/live occupancy: growing retained set;
- heap near max, low reclaim, repeated long collections: exhaustion or oversized live set;
- stable heap, rising RSS: direct/native/thread/metaspace/mapping hypothesis;
- rising loaded classes and metaspace after redeploy-like activity: class-loader retention hypothesis;
- queue depth and memory rising together: downstream throughput or backpressure failure.

A single sawtooth graph cannot identify an owner. Compare the same metric under comparable load and collector phase. Concurrent collectors can report occupancy at phases that do not correspond to a stop-the-world full collection.

Allocation pressure versus retention

Allocation pressure is often caused by repeated parsing, boxing, temporary collections, copying, oversized buffers, logging arguments, or retry amplification. JFR allocation samples and an allocation profiler identify hot allocation stacks. Optimize only allocations that materially affect GC CPU, pause, or capacity.

Retention analysis asks why objects remain reachable. Class histograms taken at comparable lifecycle points show class growth. A heap dump enables dominator trees and paths to roots. Look for the first unexpected long-lived owner, not merely the largest byte array. A million value objects may be correctly owned by one accidental map.

Histogram growth is a lead. Class counts can rise because traffic or legitimate state rises. Heap-dump retained sizes depend on the snapshot and tool model. Confirm the suspected owner in code and with a reproducible lifecycle test.

Common heap-retention patterns

- unbounded maps, caches, deduplication sets, queues, and retry histories;
- entries with expiration timestamps but no active eviction;
- `ThreadLocal` values not removed on pooled threads;
- listeners, callbacks, observers, and scheduled tasks never deregistered;
- static registries retaining application or class-loader objects;
- keys whose equality mutation prevents normal removal;
- a small view or lambda retaining a large backing object graph;
- completed futures or request contexts retained by diagnostics or metrics labels;
- class loaders retained by threads, drivers, caches, or shutdown hooks;
- weak-reference designs whose values retain keys indirectly.

Weak references are not a general cache policy. Their reclamation follows reachability and collector timing, not a service-level capacity contract. Prefer explicit maximum size, weight, expiry, admission, and observability.

Non-heap and native growth

Each platform thread consumes native stack and VM metadata; thread explosion can exhaust address space or native allocation before heap fills. Direct buffers use native memory and can be retained by Java references even though their bytes are outside the heap. Metaspace grows with classes and class loaders. JIT code cache, JNI libraries, memory maps, TLS/native libraries, and allocator fragmentation also contribute to RSS.

Correlate thread counts, direct-buffer pools, class loading, HotSpot NMT categories when pre-enabled, OS mappings, and library metrics. NMT does not cover every native allocation and RSS includes shared/resident accounting that does not match NMT totals.

OutOfMemoryError is a family of symptoms

Messages can point toward Java heap, GC overhead, metaspace, direct-buffer reservation, native-thread creation, or array-size limits. Message wording and triggering policy are implementation-specific. Preserve the exact exception, JVM logs, process limit, heap/native metrics, thread count, and allocation context.

Do not assume catch-and-continue is safe. The process may lack memory to log, allocate a response, or restore invariants. Keep emergency handling minimal, avoid large allocations, shed traffic, and use supervisor restart/failover policy. `–XX:+HeapDumpOnOutOfMemoryError` is a HotSpot option that may aid analysis but needs secure disk capacity and may fail.

Collector-neutral GC diagnosis

Ask whether GC is the cause of impact or reacting to allocation/load. Correlate pause windows with request latency, GC CPU with application CPU, allocation with traffic, and post-cycle live set with heap capacity. A pause at the same time as latency is evidence; its duration and affected requests establish contribution.

Current HotSpot distributions commonly offer throughput-oriented, region-based, and low-pause collectors. Defaults and availability depend on JDK/vendor/platform. Collector selection changes trade-offs among throughput, pause, footprint, CPU, and headroom. It cannot collect reachable objects or make an unbounded queue bounded.

> **HotSpot note:** G1 is a common default in mainstream 64-bit HotSpot server configurations for Java 17 and 21, but verify `VM.flags` or startup logs. ZGC, Generational ZGC modes, Shenandoah availability, region/humongous thresholds, and logging fields are release and vendor sensitive.

A safe incident playbook

1. Protect users: shed load, stop nonessential producers, fail over, or restart a replica according to policy.
2. Record timeline, release, traffic, limits, JVM/collector flags, exact OOM or pause symptoms.
3. Classify heap versus native using existing metrics.
4. Capture bounded JFR and GC logs if already available; take repeated histograms at comparable points.
5. Take a heap dump only after disk, pause, replica, permissions, and sensitive-data checks.
6. Analyze dominators and shortest/meaningful paths to roots; map the owner to code and lifecycle.
7. Reproduce with a bounded load test and assert cardinality or baseline recovery.
8. Fix ownership/capacity before tuning, then canary with rollback criteria.

Never invoke full GC solely to make a graph look clean without recording pre-GC evidence. `System.gc()` is a request whose treatment depends on JVM flags and collector; it is not a portable diagnostic barrier.

Tuning in the right order

First reduce unwanted retention and overload. Next set a realistic process/container budget that includes heap, metaspace, direct/native memory, thread stacks, code cache, agents, and safety margin. Then choose heap size and collector from measured objectives. Finally adjust a small number of documented flags, one hypothesis at a time.

`-Xms`, `-Xmx`, percentage-based container sizing, pause targets, and collector flags are HotSpot/vendor options. Large heaps provide burst headroom but increase footprint and some diagnostic costs. Small heaps collect more frequently. Setting minimum equal to maximum can improve predictability but commits a capacity choice and is not universally best.

TRACE IT | Worked Java example

This event bus makes listener lifetime explicit:

JAVA (continued 1/2)

```java
import java.util.ArrayList;
import java.util.List;
import java.util.function.Consumer;

record Event(String name) {}

final class RequestContext {
    private final byte[] scratch = new byte[1_000_000];
    private int eventsSeen;

    void onEvent(Event event) {
        scratch[Math.floorMod(eventsSeen++, scratch.length)] =
                (byte) event.name().length();
    }
}

final class EventBus {
    interface Registration extends AutoCloseable {
        @Override void close();
    }

    private final List<Consumer<Event>> listeners = new ArrayList<>();

    Registration subscribe(Consumer<Event> listener) {
        java.util.Objects.requireNonNull(listener);
        listeners.add(listener);
        return new Registration() {
```

JAVA (continued 2/2)

```java
            private boolean closed;

            @Override
            public void close() {
                if (!closed) {
                    listeners.remove(listener);
                    closed = true;
                }
            }
        };
    }

    void publish(Event event) {
        List.copyOf(listeners).forEach(listener -> listener.accept(event));
    }
}

public final class ListenerLifecycleDemo {
    public static void main(String[] args) {
        EventBus bus = new EventBus();
        RequestContext context = new RequestContext();
        try (EventBus.Registration ignored = bus.subscribe(context::onEvent)) {
            bus.publish(new Event("request-complete"));
        }
    }
}
```

The example is single-threaded. A production bus needs a concurrency policy, exception isolation, and documented behavior when a listener removes itself during publication. The lifecycle idea remains: subscription returns an idempotent handle that owners must close.

TRACE IT | Execution or memory walkthrough

During subscription, the method reference refers to `context`. The event bus stores that listener in its list:

```text
live bus -> listeners -> method-reference object -> RequestContext -> 1 MB byte[]
```

If the bus is application-scoped and registration is never removed, the request context remains reachable after the request ends. Repeating this pattern creates an approximately linear retained-set slope. The byte array is not the root cause; the unexpected listener ownership is.

The try-with-resources block closes the registration. `close` removes the same listener reference, breaking the path. After `main` no longer uses `context` and a future collection occurs, the context and byte array are eligible for reclamation. Eligibility does not promise immediate collection or memory return to the operating system.

`List.copyOf` in `publish` creates a snapshot of listener references so a callback cannot directly disrupt that iteration through the original list. It adds allocation and is not thread-safety. Alternative policies have different costs.

Complexity and performance

For `n` listeners, publication is `O(n)` time and the snapshot is `O(n)` temporary references. Registration append is amortized `O(1)`; removal from an array list is `O(n)`. If registration churn or listener count is high, a different representation may be justified, but it must preserve lifecycle and concurrency semantics.

Memory incident costs scale differently:

Action	Time/space tendency	Primary risk
allocation sample	sampled event cost	attribution precision
histogram	heap-object traversal/aggregation	pause and shallow-only view
heap dump	`O(heap objects + data)`	pause, disk, secrets
offline dominator analysis	roughly graph-scale, tool-dependent	analyzer memory and time
increasing heap	more headroom	larger footprint, delayed failure
bounding a queue/cache	bounded steady storage	eviction/rejection semantics

An unbounded collection is `O(n)` space where `n` may be lifetime traffic. A capacity limit turns it into `O(capacity)` but forces an explicit behavior at the boundary: block, reject, evict, spill, or degrade.

WATCH OUT | Edge cases and common mistakes

- Calling any high memory usage a leak without comparing post-collection baselines.
- Looking only at heap while process RSS or native threads grow.
- Treating a large class histogram row as the retaining owner.
- Taking one heap dump after traffic changed and inferring a slope.
- Increasing heap repeatedly while leaving cache or queue cardinality unbounded.
- Forcing full GC during the incident before preserving allocation and occupancy evidence.
- Assuming weak keys solve retention when values reference keys or cleanup lags.
- Forgetting `ThreadLocal.remove` on pooled threads.
- Adding eviction without defining what happens to correctness on eviction.
- Tuning a pause target without measuring throughput, CPU, and headroom effects.
- Using old collector flags that are ignored, deprecated, or removed on the target JDK.
- Ignoring diagnostic artifact sensitivity and disk consumption.
- Expecting memory to return immediately to the OS after objects become unreachable.
- Catching `OutOfMemoryError` and continuing normal request processing.

Production engineering notes

Put bounds and ownership on every long-lived collection, executor queue, cache, registry, and per-tenant metric label. Emit current size, maximum, eviction/rejection, oldest age, and key cardinality. Expiration needs an active maintenance policy and tests using controllable time.

Close registrations, scheduled tasks, streams, class-loader-owned executors, and thread-local state at lifecycle boundaries. Use load tests that repeat create/use/destroy cycles and assert that post-cycle cardinality or heap baselines stabilize. A one-request unit test will not reveal lifetime leaks.

Maintain memory margin below the container limit. Account for heap plus non-heap components and traffic bursts. Monitor both JVM and cgroup/host views. Configure OOM handling, secure diagnostic paths, restart policy, and traffic failover before failure.

Keep GC logging/JFR settings and parsers compatible with each supported JDK. Compare collector changes with the same workload and objectives. Roll out canaries and retain rollback capacity; a lower pause percentile may cost enough CPU to reduce total service capacity.

TRY IT | Interview questions and model answers

How can Java have a memory leak with garbage collection?

The collector removes unreachable objects. A leak occurs when objects remain reachable through an unintended long-lived path, such as an unbounded cache, listener registry, thread local, or class loader.

How do you distinguish high allocation from a leak?

High allocation shows rapid object creation and frequent collection, but post-collection live occupancy stabilizes. A retention leak tends to show a rising comparable post-collection baseline and growing owners across repeated histograms or dumps.

What would you inspect in a heap dump?

Dominators, retained size, growing classes, and paths from suspect objects to GC roots. I seek the first unexpected owner and verify its lifecycle in code rather than blaming the largest leaf array.

Why might RSS grow while Java heap is flat?

Thread stacks, direct buffers, metaspace/class loaders, code cache, JNI or native libraries, mappings, or allocator behavior. Correlate NMT where enabled with OS maps and subsystem metrics.

Should you increase `-Xmx` during an incident?

It can provide temporary headroom if process limits allow, but may delay an unbounded failure and increase footprint or diagnostic cost. Preserve evidence, define the budget, and fix ownership or capacity when that is the cause.

How do you tune GC?

Define pause, throughput, footprint, and CPU goals; fix retention and overload; size the whole process; establish a baseline; then change one supported setting or collector and test representative load before canarying.

TRY IT | Exercises

1. Draw the root path for a pooled thread retaining a `ThreadLocal` request buffer. Show the correct `try/finally` cleanup.
2. Given stable heap, growing RSS, and increasing thread count, design an evidence and mitigation plan.
3. Design bounds for a retry queue, including rejection, durability, metrics, and tenant fairness.
4. Extend the event bus with thread safety without holding a lock while callbacks execute. State snapshot and removal semantics.
5. Compare two histograms five minutes apart and list reasons class growth might be legitimate.
6. Build a canary plan for changing collectors, including latency, throughput, CPU, footprint, and rollback thresholds.

RECAP | Chapter summary

Managed-memory leaks are unwanted reachability. Diagnose them by distinguishing allocation rate, live-set growth, and non-heap process memory, then following retention paths to an unexpected owner. Stabilize service and preserve low-cost evidence before invasive dumps. Bounded collections and explicit lifecycle handles prevent more failures than collector flags. GC tuning begins only after workload, ownership, whole-process memory, and objectives are understood, and every result is specific to the tested JVM and deployment.

TRY IT | Revision checklist

- [ ] I distinguish allocation pressure, intended live data, heap leak, native growth, and resource leak.
- [ ] I correlate post-collection occupancy, allocation, GC CPU/pause, RSS, threads, classes, and queues.
- [ ] I understand shallow size, retained size, dominators, roots, and comparable snapshots.
- [ ] I can execute a staged incident playbook with safe dump criteria.
- [ ] I can identify listener, thread-local, cache, queue, key, and class-loader retention patterns.
- [ ] I budget heap and native components below process/container limits.
- [ ] I tune one measured hypothesis at a time and validate with representative load.
- [ ] I treat collector behavior, flags, OOM messages, and counters as vendor/version specific.

SDE-2 CORE CHAPTER

Chapter 4 - JVM Tools, Flags, and Incident Commands

This appendix is a field checklist for OpenJDK HotSpot-style deployments. Tool availability, exact output, attach permissions, overhead, container behavior, and flags vary by release and distribution. Treat every command as version-specific and rehearse it in a safe environment. Prefer evidence collection with understood overhead over speculative tuning.

Identify the runtime before interpreting it

Capture the executable, release, vendor, process arguments, and container limits:

TEXT

```
java -version
java -XshowSettings:vm -version
jcmd <pid> VM.version
jcmd <pid> VM.command_line
jcmd <pid> VM.flags
jcmd <pid> VM.system_properties
jcmd <pid> VM.info
```

Also record:

- wall-clock time, timezone, host/pod identity, deployment version, and incident interval;
- CPU quota and throttling, memory limit, resident memory, swap policy, and OOM-kill evidence;
- request rate, concurrency, error rate, latency percentiles, dependency health, and recent changes;
- heap sizing, collector, thread count, file descriptors, connection-pool limits, and native-library versions.

Without runtime identity and workload context, two similar-looking tool outputs can imply different mechanisms.

Evidence-first incident sequence

A conservative sequence is:

1. Establish user-visible symptoms and exact time bounds.
2. Correlate application and infrastructure metrics before attaching a tool.
3. Record runtime identity, arguments, limits, and recent deployments.
4. Collect several lightweight thread and heap summaries rather than one isolated sample.
5. Start a bounded JFR recording if its overhead is acceptable and existing continuous recordings are unavailable.
6. Escalate to heap dumps or more invasive diagnostics only with disk, pause, privacy, and operational risks understood.
7. Preserve raw artifacts and command lines; analyze copies.
8. Change one causal control at a time, define success/rollback criteria, and keep monitoring guardrails.

Restarting may restore service but destroys in-process evidence. When safety permits, collect the smallest decisive artifact first.

jcmd command map

List local JVMs and supported diagnostic commands:

TEXT

```
jcmd -l
jcmd <pid> help
jcmd <pid> help <command>
```

Common commands:

Goal	Representative command	Interpretation note
thread dump	`jcmd <pid> Thread.print -l`	take several samples; one blocked stack is not a trend
class histogram	`jcmd <pid> GC.class_histogram`	count/bytes are shallow; command may affect the process
heap overview	`jcmd <pid> GC.heap_info`	collector-specific summary, not a leak proof
heap dump	`jcmd <pid> GC.heap_dump filename=/safe/path/heap.hprof`	large and potentially sensitive; check command help/version
class-loader stats	`jcmd <pid> VM.classloader_stats`	useful for loader/metaspace growth
native memory	`jcmd <pid> VM.native_memory summary`	requires NMT enabled at startup
compiler state	`jcmd <pid> Compiler.codecache`	exact command set varies
system counters	`jcmd <pid> PerfCounter.print`	implementation counters need release context
start JFR	`jcmd <pid> JFR.start name=incident settings=profile duration=5m filename=/safe/path/incident.jfr`	quote/adjust syntax for shell and release
inspect recordings	`jcmd <pid> JFR.check`	lists active recordings
dump JFR	`jcmd <pid> JFR.dump name=incident filename=/safe/path/snapshot.jfr`	preserves current data
stop JFR	`jcmd <pid> JFR.stop name=incident`	optionally specify output per command help

Some diagnostic commands are marked with impact levels in `jcmd help`. Read them. A production-safe action depends on heap size, allocation rate, storage, pause objectives, and incident severity.

Thread diagnosis

Collect repeated dumps several seconds apart:

TEXT

```
jcmd <pid> Thread.print -l > threads-01.txt
# wait while preserving the incident interval
jcmd <pid> Thread.print -l > threads-02.txt
jcmd <pid> Thread.print -l > threads-03.txt
```

`jstack -l <pid>` is a familiar alternative when available; `jcmd` is generally the preferred HotSpot diagnostic entry point.

Look for:

- the same runnable stacks consuming samples, correlated with process/thread CPU;
- many threads waiting for one monitor, lock, connection pool, queue, or future;
- deadlock reports and an actual resource cycle;
- executor queues growing while workers block downstream;
- repeated socket/database waits aligned with dependency latency;
- virtual-thread pinning or carrier scarcity only after confirming release-specific evidence;
- shutdown threads waiting on tasks that ignore interruption.

Thread state names are snapshots. `RUNNABLE` can include native I/O or code not currently executing on a CPU. `WAITING` is not automatically a problem. Ask whether the state persists and whether it explains the service symptom.

Java Flight Recorder

JFR records time-correlated JVM and application events with configurable detail. Prefer a continuous, bounded recording established before the incident when organizational policy allows it.

Command-line startup examples:

```text
java -XX:StartFlightRecording=filename=service.jfr,dumponexit=true,maxsize=512m,maxage=2h ...
java -XX:FlightRecorderOptions=stackdepth=128 ...
```

Attach example:

```text
jcmd <pid> JFR.start name=incident settings=profile delay=10s duration=5m filename=/safe/path/incident.jfr
```

Inspect in JDK Mission Control or with the `jfr` command where available:

```text
jfr summary incident.jfr
jfr print --events jdk.CPULoad,jdk.GarbageCollection incident.jfr
```

Correlate events instead of ranking isolated hot methods. Useful relationships include allocation pressure -> GC work -> pause or concurrent CPU -> request latency; monitor contention -> blocked duration -> endpoint; socket reads -> remote host -> pool occupancy; and compilation/deoptimization -> phase change -> latency.

Recording settings and event names evolve. A profile recording can be more expensive than the default configuration. Measure overhead for the selected settings and workload.

GC and safepoint logging

Unified logging is the modern mechanism:

```text
-Xlog:gc*:file=/safe/path/gc.log:time,uptime,level,tags:filecount=10,filesize=50m
-Xlog:safepoint:file=/safe/path/safepoint.log:time,uptime,level,tags
-Xlog:gc+heap=debug,gc+age=trace:file=/safe/path/gc-detail.log:time,uptime,tags
```

Validate selectors for the exact JDK:

```text
java -Xlog:help
```

Questions to answer from GC evidence:

- Is allocation rate rising, or is the retained live set rising?
- Does old occupancy return to a stable baseline after a complete relevant cycle?
- Are pauses caused by evacuation work, reference processing, class unloading, humongous allocations, or another phase?
- Is concurrent collector work receiving enough CPU under the container quota?
- Is the heap undersized for the live set and allocation burst, or oversized in a way that harms objectives?
- Do latency spikes align with GC at all?

Never conclude "memory leak" merely because used heap rises between collections. A leak is unintended retained reachability. Use a time series and a retention analysis.

Heap histograms and dumps

A class histogram is a triage view:

```text
jcmd <pid> GC.class_histogram
```

Compare multiple histograms under similar collection conditions. Shallow bytes do not reveal who retains objects. A large legitimate cache can rank first without being a leak, while a small retaining root can keep a huge graph alive.

Heap dumps:

```text
jcmd <pid> GC.heap_dump filename=/safe/path/heap.hprof
```

Before dumping, verify:

- enough disk for approximately heap-scale output plus safety margin;
- write path and container persistence;
- pause and I/O impact for this release/heap;
- secrets, personal data, credentials, payloads, and retention policy;
- secure transfer and access controls;
- whether an existing dump-on-OOME policy is more appropriate.

Analyze dominator trees, retained size, paths to GC roots, duplicate values, and class-loader ownership. Compare a suspect population to expected workload state and lifecycle.

`jmap` offers historical heap commands, but prefer supported `jcmd` commands for the deployed JDK. Avoid force/SA attachment unless standard attachment is impossible, the risks are understood, and the incident justifies it.

Native memory and process RSS

The Java heap is only part of process memory. Other consumers include metaspace, code cache, thread stacks, direct/mapped buffers, GC/compiler structures, JNI/native libraries, allocator fragmentation, and observability agents.

Native Memory Tracking must be enabled at startup:

```text
-XX:NativeMemoryTracking=summary
```

Then:

```text
jcmd <pid> VM.native_memory baseline
jcmd <pid> VM.native_memory summary.diff scale=MB
jcmd <pid> VM.native_memory detail scale=MB
```

NMT has overhead, does not account for every native allocation, and its category names are implementation details. Correlate it with OS/container RSS, mapped memory, thread count, and native-library metrics.

A container OOM kill with no Java `OutOfMemoryError` often means the total process crossed the cgroup limit before the JVM could report a heap failure. Compare committed heap, live heap, native categories, stacks, direct buffers, sidecars, and limit/headroom.

Class loading and metaspace

Useful evidence:

```text
jcmd <pid> VM.classloader_stats
jcmd <pid> VM.classloaders
jcmd <pid> GC.class_histogram
-Xlog:class+load=info,class+unload=info
```

Metaspace growth can follow legitimate dynamic class generation, redeployment leaks, proxy churn, scripting engines, or class loaders retained by threads, caches, logging frameworks, JDBC drivers, or static registries. A class can unload only when its defining loader and associated classes are unreachable and the JVM performs relevant collection work.

Track loaded/unloaded class counts and group suspect classes by defining loader. Fix the retaining lifecycle rather than simply raising a metaspace limit.

Compiler and code-cache evidence

Warm-up, compilation, deoptimization, and code-cache pressure can create phase-dependent latency. Representative diagnostics include JFR compilation events, unified compiler logging selected for the release, and commands advertised by:

```text
jcmd <pid> help | grep -i compiler
```

Diagnostic flags such as detailed compilation output can be noisy and version-specific. Do not enable them globally in production without a test. A method that appears hot in interpreted execution may later be compiled; a microbenchmark that omits warm-up can measure compilation rather than steady-state work.

Heap and collector flags

Common controls:

Goal	Representative option	Caution
initial heap	`-Xms<size>`	committing behavior and container headroom matter
maximum heap	`-Xmx<size>`	leave room for native memory and sidecars
choose G1	`-XX:+UseG1GC`	default on many modern HotSpot configurations, but verify
choose ZGC	`-XX:+UseZGC`	capabilities and generational modes vary by release
pause target	`-XX:MaxGCPauseMillis=<ms>`	a heuristic goal, not an SLA
dump on heap OOME	`-XX:+HeapDumpOnOutOfMemoryError`	secure and provision dump path
dump location	`-XX:HeapDumpPath=/safe/path`	ensure persistence and free space
exit on OOME	`-XX:+ExitOnOutOfMemoryError`	coordinate with orchestrator and evidence policy
error files	`-XX:ErrorFile=/safe/path/hs_err_pid%p.log`	ensure writable persistent location

Avoid cargo-cult flag bundles. Defaults, flag names, diagnostic status, and collector behavior change. Begin with an objective and evidence, change the smallest relevant control, and validate under a workload with production-like allocation and latency sensitivity.

Container checklist

- Confirm the JDK recognizes the actual CPU and memory limits.
- Compare `-Xmx` and total native headroom to the cgroup limit, not host RAM.
- Inspect throttled CPU time; concurrent GC and JIT work still need CPU.
- Bound direct buffers, threads, queues, caches, and pools according to the whole process budget.
- Persist crash logs, heap dumps, and JFR outside ephemeral storage when policy permits.
- Align orchestrator termination grace with Java shutdown and diagnostic time.
- Distinguish Java OOME, kernel/cgroup OOM kill, liveness restart, and operator eviction.

Symptom-to-evidence map

Symptom	First evidence	Common false lead
CPU saturation	process/thread CPU, repeated stacks, JFR execution samples	assuming any `RUNNABLE` thread is consuming CPU
long tail latency	request spans/metrics, JFR, GC/safepoint correlation, pool occupancy	blaming GC without time alignment
growing heap after GC	live-set series, histograms, heap dominators and roots	reading sawtooth growth as a leak
rising RSS with flat heap	NMT, thread/direct-buffer counts, mappings, native tools	increasing `-Xmx`
stuck shutdown	repeated thread dumps, executor/task cancellation paths	calling `System.exit` before preserving evidence
deadlock	JVM deadlock report plus resource graph	labeling ordinary lock contention deadlock
task starvation	executor active/queue metrics, worker stacks, dependency waits	adding unbounded threads
allocation storm	JFR allocation samples, GC log allocation rate, endpoint correlation	tuning pause targets before fixing churn
metaspace growth	class/loader counts, loader stats, unload logs, roots	only raising metaspace maximum
container OOM kill	cgroup event, RSS/native budget, limit history	expecting a heap dump automatically

Incident handoff template

A good incident explanation separates observation, inference, and decision. Example: "old occupancy remained above 82% after three mixed cycles" is an observation; "a retained cache is likely" is a hypothesis; "capture a bounded heap dump on one canary" is a decision with operational and privacy costs.

Record:

TEXT

```
User impact and interval:
Deployment/runtime identity:
Host or container limits:
Workload and dependency state:
Recent changes:
Thread evidence (timestamps and repetitions):
GC/safepoint evidence:
JFR recording and settings:
Heap/native evidence:
Leading hypotheses:
Evidence that supports/refutes each hypothesis:
Mitigation, risk, and rollback:
Follow-up experiment and owner:
Artifact locations and data classification:
```

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: read tool output and define metrics
- Interview Core: design benchmark and incident hypotheses
- SDE-2 Follow-up: choose discriminating evidence
- Challenge: write an end-to-end incident playbook

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: Study Step 18D - Advanced Java D: Concurrency and the Memory Model

Series index

Next book: Study Step 18F - Advanced Java F: Design, Backend, Testing, and Security

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the study steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. The same public study codes appear on the website and every PDF cover. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
01	Java Foundations for Problem Solving
02	Time and Space Complexity for Java Interviews
03A	Number Systems and Math Foundations for DSA Interviews
03B	Number Systems Interview Patterns and Rapid Revision
04	Bit Manipulation in Java
05	Loop Mastery, Patterns, and Index Calculations
06	Arrays and Array Problem-Solving Patterns
07	Strings and String Problem-Solving Patterns
08	Hashing: Maps, Sets, Frequency, and Prefix State
09	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18A	Advanced Java A: JVM and Execution
18B	Advanced Java B: Language, OOP, and Modern Java
18C	Advanced Java C: Collections, Streams, and I/O
18D	Advanced Java D: Concurrency and the Memory Model
18E	Advanced Java E: Performance, Diagnostics, and GC Incidents
18F	Advanced Java F: Design, Backend, Testing, and Security
18G	Advanced Java G: Question Bank, Study Plan, and Reference
18H	Advanced Java H: Spring Boot and REST APIs
18I	Advanced Java I: Persistence, SQL, JPA, and Caching
18J	Advanced Java J: Distributed Systems and System Design

Study Step 03 uses linked foundations (03A) and workbook (03B) books. Study Step 18 uses ten focused books (18A-18J), including a three-book backend specialist track. These suffixes keep every file focused while preserving one unambiguous route.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Study Step 18E of 18 - Part E of J. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.